Term project: Edge AI and IoT Integration

SLA-Aware Split Inference with Early Exit for IoT

Student: Nguyen Minh Chinh (411221380) **Course:** CSIE52400 Distributed System, National Dong Hwa University **Date:** October 1, 2026

1. Motivation

Integrating edge AI with the Internet of Things means bringing deep neural network (DNN) inference to devices such as cameras, drones and wearable sensors, which lack the compute and energy to run DNNs quickly on their own. Offloading raw data to the cloud or an edge server costs bandwidth, adds network latency and fails when connectivity degrades. The best place to run each part of a model therefore changes with network conditions and server load, which makes inference placement a distributed systems problem rather than a purely machine learning one.

2. Objectives

1. Build a multi-exit DNN that can be split between an IoT device and an edge server.
2. Design a runtime scheduler that jointly chooses the split point k and the early-exit confidence threshold τ from measured bandwidth and edge load, to meet a user-defined service-level agreement (SLA), e.g. "p95 latency under 100 ms, maximise accuracy".
3. Support many devices sharing one edge server, and fall back to local inference when the edge fails.
4. Compare against all-on-device, all-offloaded, split-only and exit-only variants.

3. Related work

SPINN [1] co-optimises the DNN split point and the early-exit policy at runtime so that inference adapts to changing network conditions and server load; it is the main basis for this project. Edgent [2] combines device–edge partitioning with model right-sizing to meet a latency deadline. Li et al. [4] study fine-grained partitioning when many IoT devices share edge servers, which motivates the multi-client experiments. CoEdge [3] takes a different approach, splitting a single inference across several cooperating devices; device-to-device cooperation is left as an optional extension. The survey in [5] frames split computing and early exiting as a research area.

4. System design

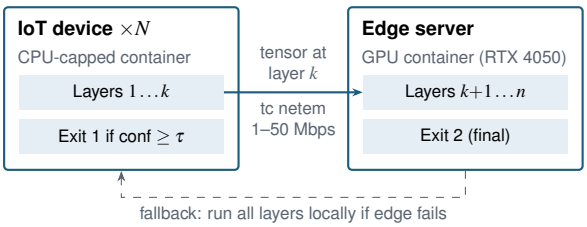


Figure 1. Device–edge pipeline; the scheduler sets k and τ together.

Offline, a profiler records per-layer compute time on each tier, per-layer output size, and each exit's accuracy and exit rate at different thresholds. Online, the scheduler reads current bandwidth and edge queue length, predicts latency and accuracy for every (k, τ) pair, and picks the pair that best meets the SLA. It re-plans when conditions change. Tensors move over gRPC; a heartbeat detects edge failure and triggers local fallback.

5. Experimental setup

The testbed is emulated on one laptop (RTX 4050, 6 GB VRAM) using WSL2 and Docker Compose. Device containers are limited to 0.25 CPU and 512 MB and pinned to separate cores to approximate a Raspberry Pi; the edge container uses the GPU. `tc netem` sets bandwidth, delay and loss per link. Workload: MobileNetV2 and ResNet-18 on Imagenette, a 10-class ImageNet subset, at 224×224, with 1–10 concurrent clients and injected edge failures.

6. Evaluation metrics

SLA satisfaction rate (share of requests meeting the target), end-to-end latency (mean and p95), throughput, top-1 accuracy, bytes transmitted per inference, device exit rate, time to adapt after a bandwidth change, and recovery time after edge failure.

7. Timeline

Weeks	Work
4–5	Literature review, proposal, Docker/GPU testbed
6–7	Baselines, CPU-cap calibration, layer profiling
8	Multi-exit model training, exit accuracy profiling
9–11	Joint (k, τ) scheduler, SLA modes, runtime re-planning
12	Multi-client scaling and fault tolerance
13–14	Full experiments under varied network and load
15	Buffer: catch-up, re-runs, optional extension
16	Analysis and final report
17	Demo video recording and submission

8. Expected outcomes and limitations

The joint scheduler should meet the SLA for more requests than the split-only and exit-only variants and both static baselines across a range of bandwidths and edge loads, while transmitting less data and staying within a small accuracy margin. Because all nodes share one physical host and the network is emulated, absolute numbers will differ from real deployments; energy is estimated from a model rather than measured. These threats to validity will be discussed in the final report.

[1] S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, N. D. Lane, "SPINN: Synergistic Progressive Inference of Neural Networks over Device and Cloud," in *Proc. ACM MobiCom*, 2020. doi:10.1145/3372224.3419194

[2] E. Li, L. Zeng, Z. Zhou, X. Chen, "Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing," *IEEE Trans. Wireless Communications*, vol. 19, no. 1, 2020. doi:10.1109/TWC.2019.2946140

[3] L. Zeng, X. Chen, Z. Zhou, L. Yang, J. Zhang, "CoEdge: Cooperative DNN Inference with Adaptive Workload Partitioning over Heterogeneous Edge Devices," *IEEE/ACM Trans. Networking*, vol. 29, no. 2, 2021. doi:10.1109/TNET.2020.3042320

[4] H. Li, X. Li, Q. Fan, Q. He, X. Wang, V. C. M. Leung, "Distributed DNN Inference with Fine-Grained Model Partitioning in Mobile Edge Computing Networks," *IEEE Trans. Mobile Computing*, vol. 23, no. 10, 2024. doi:10.1109/TMC.2024.3357874

[5] Y. Matsubara, M. Levorato, F. Restuccia, "Split Computing and Early Exiting for Deep Learning Applications: Survey and Research Challenges," *ACM Computing Surveys*, 2022. doi:10.1145/3527155